

Front-End Library Assignment (React + TypeScript + MUI)

This assignment assesses your ability to design a small, accessible, and well-typed React component library using React, TypeScript, and Material UI (MUI). The goal is to show code quality, design consistency, and developer experience within a manageable scope.

Make sure you are proud of the code you implement for this assignment. Architectural choices are more important than implementing all the features. Quality is key. Make sure each part of your code is behaving correctly and not susceptible to regression issues. If there is anything unclear in the assignment, note your assumptions; these can be discussed later.

Objectives

1. Build a small, themed UI foundation using MUI.
2. Demonstrate TypeScript proficiency (strict types, inference-friendly props, no any).
3. Apply accessibility best practices (roles, focus, keyboard, contrast).
4. Deliver maintainable, readable, and testable code.

Scope (feasible for ~4 hours)

Implement three core components and a basic theming setup.

1. ThemeProvider

- Light and dark mode.
- Define and apply a color palette and spacing tokens.
- Expose a small API (`useThemeMode` or similar) to toggle theme.

2. Autocomplete with Search & Match Highlighting

Features

- Search input with debounced filtering (configurable; default 200 ms).
- List with match highlighting (bold or colored spans) for query substrings across `name` and `code` (e.g. `"AMS"`).

- Keyboard support: **↑/↓** to navigate, **Enter** to select, **Esc** to clear/close.
- Empty state, loading state, and no-results message.

Optional (not required)

- Async data hook: `onSearch(query) => Promise<Option[]>` (fallback to local filter if not provided).
- Grouping by **group** (e.g. country or region) with sticky group headers.

3. Multi-Origin-Destination Filter (Stations)

Purpose: Select one or more origins and one or more destinations from a station list; emit a normalised filter payload.

Features

- Two inputs ("Origin(s)", "Destination(s)"), each using the Autocomplete in multi-select mode with chips.
- Support add/remove items via keyboard (**Backspace** to remove last chip, **Delete** to remove focused chip).
- Clear-all action per field.
- Handles large lists efficiently (virtualised list optional; basic list acceptable).

Optional (not required)

- A toggle to "mirror destinations = origins" for symmetric routes.

Deliverables

Deliver a single-page demo app (no routing needed) that renders all three components together in a realistic scenario — for example, a journey search filter panel. The page must include a theme toggle (light/dark) so both modes can be evaluated. This is the primary way to demonstrate how the components work together.

1. Source code in a public GitHub repository.
2. A demo app as described above.
3. **README** describing:
 - How to run and build the project.

- Architecture overview (1–2 paragraphs).
- Design/theming approach and accessibility notes.

Setup & Constraints

- Implement setup yourself using Vite + React + TypeScript + MUI.
- You may use a public template to save setup time (MUI or Vite examples).
- Do not include backend code or build pipeline tooling beyond what is required to run and build.
- Keep dependencies light; each added dependency should be clearly motivated.

Suggested public bases (optional):

- [MUI Vite + TS example \(official\)](#)
- [Vite React TS starter](#)

Submission

- Submit a public GitHub link with setup instructions.
- Include a brief **Architecture Notes** section in the **README**.
- Optionally, record a short (1–2 min) screen capture of the demo app.